SDK 文档

未发布作品。© 2023 Siemens 本材料包含 Siemens Industry Software, Inc.、其子公司或其关联公司（统称"Siemens"）的商业秘密或其他保密信息，或其许可方的商业秘密或其他保密信息。访问和使用此信息须严格遵守客户与 Siemens 适用协议中的规定。未经 Siemens 事先明确书面许可，不得将本材料复制、分发或在客户设施以外披露，也不得以任何未经 Siemens 明确授权的方式使用。 本文件仅供信息和指导之用。Siemens 保留随时更改本出版物中规格和其他信息的权利，读者在所有情况下应咨询 Siemens 以确定是否有任何更改。本文件中关于产品、特性或功能的表述构成技术信息，而非保证或承诺，不会引起 Siemens 的任何责任。Siemens 不提供任何保证，包括但不限于适销性和特定用途适用性的默示保证。特别地，Siemens 不保证产品的运行不会中断或无错误。 管辖 Siemens 产品销售和许可的条款和条件在 Siemens 与其客户的书面协议中规定。Siemens 的最终用户许可协议和通用合同协议可在以下地址查看：https://www.sw.siemens.com/en-US/sw-terms/ 商标：本文中使用的商标、徽标和服务标志（"商标"）为 Siemens 或其他方的财产。未经 Siemens 或商标所有者的事先书面同意（视情况而定），任何人不得使用这些商标。本文中使用第三方商标并非试图表明 Siemens 是产品的来源，而是指明来自特定第三方或与其关联的产品。Siemens 商标列表可在以下地址查看：https://www.plm.automation.siemens.com/global/en/legal/trademarks.html。注册商标 Linux® 根据 Linus Torvalds（该商标全球所有者）的独家被许可方 LMI 的再许可使用。

关于 Siemens Digital Industries Software Siemens Digital Industries Software 是全球领先的产品生命周期管理（PLM）软件和服务提供商，拥有 700 万个许可席位和全球 71,000 家客户。Siemens Digital Industries Software 总部位于德克萨斯州普莱诺，与企业协作提供开放解决方案，帮助他们将更多创意转化为成功的产品。有关 Siemens Digital Industries Software 产品和服务的更多信息，请访问 https://www.siemens.com/plm。 支持中心：https://www.support.sw.siemens.com 文档反馈：https://www.support.sw.siemens.com/doc_feedback_form

---

目录

1 SDK 内容 2 关于 SDK 3 Polarion API 使用说明 4 Polarion Java API 4.1 IExportManager 示例 4.2 环境要求 4.3 工作区准备 4.4 部署到已安装的 Polarion 4.5 从工作区执行 5 Polarion Web Services API 5.1 Web Services 环境要求 6 示例 6.1 Java API 示例 6.1.1 Servlet（小服务程序）示例 6.1.2 工作流函数与条件示例 6.1.3 任务（Job）示例 6.1.4 通知扩展示例 6.1.5 表单扩展示例 6.1.6 枚举工厂示例 6.1.7 管理扩展示例 6.1.8 自定义导出器示例 6.2 Web Services 示例 6.2.1 工作项导入示例 6.2.2 提交前钩子（Pre-commit hook）示例 7 Polarion Java API 文档（用于 Polarion 扩展） 8 Hivedoc 9 Web Services 文档 10 数据库 11 自定义字段类型的 Java 类

2 关于 SDK

Polarion 软件开发工具包（Software Development Kit，SDK）是一组有用的文档和库，可帮助您学习如何访问 Polarion 系统——既可以通过 Java 应用程序编程接口（Application Programming Interface，API）作为扩展使用，也可以通过 Web Services 远程访问。建议从阅读 SDK 文档开始，特别是当您希望开发以下内容时：

- 特殊 Polarion 功能的扩展，如工作流函数或条件
- 用于主页（Home）或仪表盘（Dashboard）的自定义 Servlet（小服务程序）
- 在 Polarion 中运行的 Tomcat 应用程序，例如操作工作项（Work Items）、遍历存储库，或在向存储库提交某些内容之前检查某些条件（使用 Web Services）

3 Polarion API 使用说明

Polarion Web Services

使用 Polarion Web Services 可以执行的操作：

- 读取、修改或创建新的工作项
- 列出项目
- 管理用户
- 关联工作项
- 按查询、可用操作或负责人列出工作项
- 列出构建（Builds）
- 创建、修改和复用模块（Modules）

使用 Polarion Web Services 无法执行的操作：

- 编辑 Polarion 本身的配置。例如，无法开启/关闭电子邮件通知。
- 创建构建
- 编辑门户（Portals）或工作流（Workflow）
- 编辑类似的设置

4 Polarion Java API

在同一 Java 虚拟机（Java Virtual Machine，JVM）内部对 Polarion 服务器的访问由多个服务提供。最重要的几个服务列在下方。（有关其他可用服务，请参阅 Polarion Java API 文档。）

服务接口	功能说明
ISecurityService	认证和授权任务的入口点。SecurityService 管理用户、角色、关系和权限。
ITrackerService	与追踪器（Tracker）相关功能的主要入口点。这些功能按领域划分为各个管理器（Manager），提供以下功能：搜索、读取、创建、修改和关联工作项及其属性，如工作记录（Work Records）、附件（Attachments）、评论（Comments）和时间点（Time Points）。
ITransactionService	此服务提供了一种将更改持久化到存储库的机制，包含多种方式，如异常包装和在发生错误时回滚的能力，或执行提交操作。
IDataService	提供对可持久化数据对象的操作，如搜索、保存、解析以及通过修订版本查看其历史记录。
IRepositoryService	在分层（文件夹-文件）结构中存储任意文件。让您能够操作存储库（Repository）。
IExportManager	允许创建和删除导出及模板。支持以下格式的导出：CSV、MS Office Excel 表格和模板、MS Office Word 模板、MS Project。（请参阅后续章节中的示例。）
IDocumentsManager	此接口包含用于操作以文档格式（Microsoft Word 和 Excel）存储的工作项以及文档本身的特殊方法。
IBuilderService	搜索和运行构建。
IContextService	此服务在存储库（Repository Service）之上构建上下文的逻辑结构，供应用程序的其余部分使用。上下文层次结构是一个树形结构，可以遍历并随时间动态变化。
IAnnouncerService	通过指定协议（如 SMTP）发送公告。
IContributionManager	此接口发现现有的工作流条件（Conditions）、函数（Functions）和验证器（Validators）及其实例化。可用于需要了解哪些条件、函数或验证器可用的客户端（例如工作流编辑器）。
IWorkFlowManager	处理与工作流相关的功能，包括工作项的工作流转换和工作流配置的操作。
IJobUnitFactory	用于创建新任务的服务。
IShutdownService	关闭 Polarion 服务器的服务。

4.1 IExportManager 示例

```
IExportManager em = trackerService.getExportManager();
IExporterDescriptor desc = em.getExporterDescriptor(null);
```

（例如：`IExporterDescriptor exporter = exportMgr.getExporterDescriptor(IExportManager.EXP_WORD_TEMPLATE);`）

```
IExportTemplate template = em.getTemplate(desc, null, null);
```

（`IExportTemplate` 是导出模板。导出模板可以由导出管理器管理并作为文件存储在存储库中，也可以是使用导出模板工厂为特定导出创建的自定义模板。）

```
IExportConfiguration conf = new ModuleExportConfiguration(module, null, null,
template, null);
```

下图展示了 params 参数的示例。

```
IExport ex = em.startExport(desc, null);
```

上面创建的 "conf" 可以作为第二个参数提供给 startExport 方法。

```
InputStream str = ex.getResult();
```

`ModuleExportConfiguration` 构造函数的 params 参数示例。更多信息请参阅 Javadoc（`Polarion\polarion\SDK\doc\javadoc`）。

4.2 环境要求

开发环境

- Eclipse IDE for Enterprise Java Developers 或其他包含 Eclipse Plug-in Development Environment 的 Eclipse IDE。（操作路径：Help > Install New Software… > Install Eclipse Plug-in Development Environment > Restart Eclipse）
- Eclipse Temurin™ 17 (LTS) by Adoptium，用于构建和运行代码。

4.3 工作区准备

要开始开发 Polarion Java API 插件（Plugin），首先需要执行以下步骤：

1. 启动 Eclipse，然后选择 Window > Preferences...
2. 在出现的对话框中，选择 Plug-In Development > Target Platform。
3. 点击右侧的 Add 按钮。
4. 保持选中 Nothing: Start with an empty target definition 选项，然后点击 Next。
5. 输入名称并点击 Add。
6. 选择 Directory 并点击 Next。
7. 点击 Browse，选择 `C:\Polarion\polarion` 文件夹（Windows）或 `/opt/polarion/polarion`（Linux）。（即 plugins 文件夹的上一级目录。）
8. 点击 Next。
9. 将显示当前已安装的 Polarion 插件列表。点击 Finish。
10. 显示所选路径和发现的可用插件数量。确认路径正确后点击 Finish。
11. 勾选新添加路径旁边的复选框，然后点击 Apply。

4.4 部署到已安装的 Polarion

您可以两种方式将插件部署到 Polarion。第一种方式是将项目导出为 Deployable Plugins and Fragments。第二种方式将在下一节"从工作区执行"中描述。导出插件的操作步骤如下：

1. 选择 File > Export...
2. 在出现的对话框中，选择 Plug-in Development 部分的 Deployable Plugins and Fragments，然后点击 Next 按钮。
3. 勾选您的项目（例如对于 Servlet 示例为 `com.polarion.example.servlet`），并在目标目录中指定 Polarion 安装目录下的 `polarion` 文件夹（通常位于 `C:\Polarion\polarion`）。
4. 在 Options 选项卡中，确保 Package plug-ins as individual JAR archives 未被勾选。点击 Finish。
5. 因为这是一个新的 Polarion 插件扩展，您必须重启 Polarion 服务器。

注意： 由 Polarion 加载的 Servlet 会被缓存在 `[Polarion_Home]\data\workspace\.config` 中。如果在部署 Servlet 扩展（插件）并重启 Polarion 之前未删除此文件夹，则 Servlet 可能无法正确加载，或者旧的 Servlet 会被加载。

4.5 从工作区执行

将插件部署到 Polarion 的第二种方式是从 Eclipse 工作区直接启动 Polarion。此方法的额外优势是可以在 Eclipse 中直接调试代码。

1. 选择 Run > Open Debug Configurations...
2. 创建一个新的 Eclipse 应用程序（双击 Eclipse Application）
3. 您应设置：
    - 名称（Name）为 `Polarion Server`
    - 工作区数据位置（Workspace Data Location）为 `C:\Polarion\data\workspace`（假设您的 Polarion 安装在 `C:\Polarion\`）
    - 运行应用程序（Run an application）设为 Program to Run 部分中的 `com.polarion.core.boot.app`
4. 然后设置您的 Runtime JRE。在第二个 "Arguments" 选项卡中，设置以下参数：

    Program Arguments 部分：

    Windows： `-os win32 -ws win32 -arch x86 -appId polarion.server`

    Linux： `-os linux -ws gtk -arch x86_64 -appId polarion.server`

    VM Arguments 部分：

    Windows： `-Xms256m -Xmx640m -Dcom.polarion.home=C:\Polarion\polarion`

    Linux： `-Xms256m -Xmx640m -Dcom.polarion.home=/opt/polarion/polarion -Dcom.polarion.propertyFile=/opt/polarion/etc/polarion.properties`
5. 您现在必须根据您的安装情况更改 Polarion 服务器的参数。您可以参考以下截图检查设置。
6. 在第三个 "Plug-ins" 选项卡中，确保您也选择了 "Target Platform" 插件。
7. 全选，然后点击 Validate Plug-ins 按钮。如果存在问题，请取消选中有冲突的插件。
8. 其他页面无需更改。直接点击 Debug 按钮，继续运行您的新 Polarion Server 应用程序。

---

5 Polarion Web Services API

Polarion 提供了一组文档风格的 SOAP Web Services（简单对象访问协议 Web 服务），用于外部集成。对于所有这些服务，Polarion 都提供了 WSDL（Web 服务描述语言）和 Java 客户端桩（Client Stubs）。

使用客户端桩与 Web Services 交互的常规方式如下：

```
final WebServiceFactory factory = new WebServiceFactory("http://POLARIONHOST:8888/
polarion/ws/services/");
final SessionWebService session = factory.getSessionService();
session.login(userName, password);

// 您的业务逻辑，可以从 factory 获取更多桩，如 TrackerWebService，可以在 Session Service 上
启动/结束事务等。

session.endSession();
```

更多信息请参阅 Web Services 示例。

请注意，从 WebServiceFactory 获取的桩不是线程安全的（Thread Safe）。如果多个线程并发访问单个桩实例，结果可能不可预测。您应正确同步对这些桩的访问，或者可以使用线程封闭（Thread Confinement），为每个线程使用一个桩实例。TrackerWebService 上的工厂方法每次调用都会创建一个新实例。

以下代码片段展示了如何实现线程封闭的示例：

```
public static class ThreadCallingWebService implements Runnable {

    private SessionWebService sessionService;

    public ThreadCallingWebService(WebServiceFactory factory) throws
ServiceException {
        sessionService = factory.getSessionService();
        // 获取所有需要的桩并将其存储到成员变量中，确保这些桩不会逃逸出线程。
    }

    @Override
    public void run() {
        sessionService.logIn(userName, password);
        // 使用桩执行一些操作。
    }
}
```

sessionService 作为成员变量与线程关联，因此除非变量逃逸出线程（例如引用被传递到其他上下文，使其他线程可以使用它），否则其他线程无法访问它。

5.1 Web Services 环境要求

- **Polarion Web Services 客户端** — 在 `lib/com.polarion.alm.ws.client` 中可以找到 Polarion Web Services 客户端的编译库，以及项目二进制文件，可作为 Web Services 应用程序的依赖项导入 Eclipse 工作区。该客户端的源代码在同一文件夹中以压缩形式提供。

6 示例

了解 Polarion 扩展的最佳方式是从准备好的示例中学习。我们为您提供了四个使用 Java API 部署在 Polarion 服务器中的示例。第一个示例通过创建一个函数和一个条件来扩展工作流系统。第二个示例展示了如何在存储库概览页面中嵌入您自己的 '.jsp' 页面，以描述基于相关项目属性的有用信息。第三个示例是自定义任务单元的实现，该任务检查工作项的到期日期，如果工作项延迟，则向负责人或全局邮箱发送通知。最后一个示例展示了如何通过创建新事件和新目标来扩展通知系统。

如上一节所述，当您需要在执行某些操作之前检查 Polarion 中的某些内容，但需要从独立应用程序中执行此操作时，Web Services 非常有用，在这种情况下 Web Services 是最佳解决方案。一个已实现的典型示例是在执行向存储库提交修订之前调用的钩子（Hook）（因此名为 pre-commit hook，即提交前钩子）。第二个 Web Services 示例展示了一种快速开发应用程序的方法，其中输入端有数据（例如 CSV 文件），您需要一个批处理脚本将其上传到 Polarion 中。

6.1 Java API 示例

6.1.1 Servlet 示例

SE - 简介

此示例允许您创建一个 Wiki 页面扩展，以自定义 Servlet 的形式通知用户，例如在主页或仪表盘上显示统计信息。结果将是您自己的 Servlet，包含标题和由您编写的 '.jsp' 页面表示的正文，嵌入到 Wiki 页面中。

SE - Java API 工作区准备

请参阅"工作区准备"章节。

SE - 导入示例

信息： 您必须确保您的插件是针对您的 Polarion 版本编译的。此示例包含预编译的 jar 插件。您可以基于此示例开始开发自己的插件之前将其删除。Eclipse 会确保根据您的源代码和 Polarion 版本创建新的 jar 插件。

将工作流项目示例导入工作区的步骤：

1. 选择 File > Import...
2. 在出现的对话框中，选择 General 部分的 Existing Project into Workspace，然后按 Next 按钮。
3. 按 Browse.. 按钮，选择示例目录（通常在 `C:\Polarion\polarion\SDK\examples\` ）。提交。
4. 选择 `com.polarion.example.servlet` 并按 Finish。

SE - 开发自己的插件提示

1. 首先，我们需要创建一个新的 Eclipse 插件：选择 File > New.. > Project。
2. 在出现的对话框中，选择 Plug-in Project 并按 Next 按钮。
3. 将项目名称设置为例如 `com.polarion.example.servlet`。按 Next。
4. 取消选择 Generate an activator... 按 Next。
5. 按 Finish。
6. 从同名目录中打开 MANIFEST.MF。
7. 点击第二个页面 - Dependencies，点击 Add.. 按钮。
8. 输入 `com.polarion.portal.tomcat` 并提交。
9. 重复前两个步骤，但输入 `com.polarion.alm.tracker` 并提交。
10. 在下一个页面的 'Classpath' 角落点击 New..，输入 `servlet.jar` 并提交。
11. 在下一个页面点击 Add.. 并选择 `com.polarion.portal.tomcat.webapps` 扩展点（Extension Point）并提交。
12. 在 Extension Element Details 中为新应用程序设置带有 "polarion/" 前缀的名称，例如 "polarion/example"，并将 contextRoot 设置为 `webapp`。
13. 根据图 SE-7 在 Binary Build 角落中选择文件夹和文件。（如果某些文件夹缺失，稍后可以再选择。）
14. 要检查之前的步骤，可以将新创建的文件与示例文件进行比较。
15. 点击 src 目录并选择 File.. > New > Package，作为包名称您可以粘贴 `com.polarion.example.servlet`。
16. 点击您刚刚创建的包，选择 File.. > New > Class，将名称设置为您的 Servlet 类名称，例如 `CurrentUserWorkloadServlet`。
17. 在 Superclass 行点击 Browse 并输入 `HttpServlet`。选择 OK 并按 Finish。
18. 根据图 SE-8 创建文件和目录。

SE - 部署到已安装的 Polarion

请参阅"部署到已安装的 Polarion"章节。

SE - 从工作区执行

请参阅"从工作区执行"章节。

SE - 配置

成功将插件部署到 Polarion 后，您需要在仪表盘中包含 Servlet。

转到存储库级别的 Dashboard 主题。编辑 Dashboard Wiki 页面，添加以下行：

```
1.1.1 Current User Workload
<iframe width="100%" height="200" src="/polarion/example/" frameborder="0"></
iframe>
```

保存后，您应该在页面底部看到新的 Servlet。

6.1.2 工作流函数与条件示例

WFCE - 简介

此工作流示例是一个自定义工作流函数和条件：

- **Compute Total Life Time（计算总生命周期）函数** 将计算从创建工作项到触发将状态设为已关闭（Closed）的操作之间的时间。
- **Comments Exist（评论存在）条件** 将检查工作项的状态，如果结果为正，则该操作将被允许（并可用）。

WFCE - 函数与条件的区别

工作流条件（Workflow Condition）决定是否执行某个操作。如果条件满足，则该操作可用。工作流函数（Workflow Function）在确认所做的更改后会立即执行某些操作。因此，条件总是在您开始编辑工作项之前进行测试，而函数在您完成时执行。

WFCE - 工作流函数：Compute Total Life Time

此函数的典型用途是计算从创建工作项到触发将状态设为已关闭操作之间的时间——即将工作项标记为已完成的时间。（建议将此函数用于到 Closed 状态的转换。）

此函数有一个名为 'field' 的参数，其值为自定义字段的名称。自定义字段类型必须为 'string'。结果将仅保存到此自定义字段中，字段值为从创建工作项到此操作的时间。请参阅"配置"章节了解如何在 Polarion 中设置。

WFCE - 工作流条件：Comments Exist

工作流条件的典型用途是在执行操作之前检查某些内容，例如转换到另一个状态。结果始终是布尔值（Boolean），'true' 表示成功。

此条件检查是否存在任何评论。如果至少存在一条评论，则返回 'true'。

假设我们有一个处于 'open' 状态的工作项，我们为 'Resolve and Close' 操作设置了此条件：

- 评论数量等于 0，意味着可用操作为：Accept、Resolve。
- 评论数量大于 0，意味着可用操作为：Accept、Resolve、Resolve and Close。

WFCE - Java API 工作区准备

请参阅"工作区准备"章节。

WFCE - 创建项目插件

您可以创建或导入构建项目。

WFCE - 导入示例

信息： 您必须确保您的插件是针对您的 Polarion 版本编译的。此示例包含预编译的 jar 插件。您可以基于此示例开始开发自己的插件之前将其删除。Eclipse 会确保根据您的源代码和 Polarion 版本创建新的 jar 插件。

将工作流项目示例导入工作区的步骤：

1. 选择 File > Import...
2. 在出现的对话框中，选择 General 部分的 Existing Project into Workspace，然后按 Next 按钮。
3. 按 Browse.. 按钮，选择示例目录（通常在 `C:\Polarion\polarion\SDK\examples\` ）。提交。
4. 选择 `com.polarion.example.workflow` 并按 Finish。

WFCE - 开发自己的插件

首先，您必须创建新的插件项目。填写插件属性（Plug-In Properties）并取消选中 Generate activator..。

然后打开 MANIFEST.MF，在 Dependencies 页面将 `com.polarion.alm.tracker` 设置为 Required Plug-in。您还应在 Build 页面中将 `src/` 文件夹设置为应编译到导出库中的源文件夹。

build.properties 文件内容：

```
source.. = src/
output.. = bin/
source.example-plugin.jar = src/
bin.includes = META-INF/,\
               example-plugin.jar,\
               .
```

为了让 Polarion 服务器知道您创建了新的工作流函数或条件，您必须在 src 文件夹中创建 META-INF 目录，并将 hivemodule.xml 文件放置在其中。更多信息请参阅示例中的 hivemodule.xml。

WFCE - 构建和部署

Polarion 的插件有两种基本类型。第一种是仅包含源代码的"标准"插件。第二种是"Web 应用程序"插件，或包含外部资源的插件。例如，Web 应用程序使用名为 `webapp` 的特殊目录。此目录包含将由 Web 服务器部署到 Polarion 的外部资源（网页、图片）（另请参阅 Servlet 示例，了解如何将 webapp 目录添加到 Web 服务器）。如果您想为 Polarion 开发基于 Web 的应用程序或包含外部资源的插件，必须将 webapp 或资源目录与 jar 插件分开。

这可以通过 build.properties 文件实现。您可以在 build.properties 中添加以下行：

```
bin.includes = META-INF/,\
               plugin.xml,\
               webapp/,\
               example-plugin.jar
```

可以看到，`example-plugin.jar` 是我们的 Polarion 插件，将部署在插件目录中。此插件目录还包含包含我们页面和图片的 webapp 目录。

这种部署方式的优点是您可以访问这些资源，因为它们没有直接打包在 jar 插件中。

WFCE - 部署到已安装的 Polarion

请参阅"部署到已安装的 Polarion"章节。

WFCE - 从工作区执行

请参阅"从工作区执行"章节。

WFCE - 配置

成功将插件部署到 Polarion 后，您可以开始使用新的工作流函数和条件。要检查部署是否成功，请执行以下步骤：

1. 进入管理界面（Administration），打开您要设置新函数的项目。
2. 转到 Work Items > Workflow。
3. 转到最右边的列，为特定类型的工作项创建新配置。
4. 在工作流设计器（Work Flow Designer）中，转到最后一个端口 'Actions'，点击任何操作行上的编辑图标（紫色圆圈中的白色对勾）。将显示已指定的操作详情（条件和函数）的弹出编辑器。在那里您应该能看到新的条件和函数。

选择您要设置条件或操作的操作，然后根据图像设置属性。

6.1.3 任务示例

JE - 简介

此任务示例是自定义任务单元的实现。该实现检查工作项的到期日期，如果工作项延迟，则向负责人或全局邮箱发送通知。此示例还涵盖了参数化任务单元的构建，允许您为任务定义参数并通过 IAnnouncerService API 类发送公告。

此示例还向您展示了 Polarion 的另一部分，允许您创建调度器（Scheduler）系统的扩展。调度器定期执行任务，并可以基于工作项计算一些统计数据。

JE - 什么是调度器和任务?

调度器是任务的组织者。您可以通过调度器编辑任务以定期执行或监控任务。调度器的灵感来源于 Cron 调度器，您可以使用 Cron 表达式轻松维护执行。它允许您设置每五分钟、每小时或按您喜欢的频率执行。

任务是调度器的执行单元，由调度器定期执行。任务的操作可以各不相同——从基于工作项的简单计算到发送定期构建分析报告。

JE - 逾期任务：检查逾期工作项

工作项的典型用法是允许您指定解决的到期日期。如果追踪器中有许多工作项，通知每个用户其延迟情况是相当复杂的。此任务可以通过任务轻松实现。您可以为调度器扩展您自己的任务，定期检查每个工作项是否在到期日期之前解决。然后如果发现一些延迟的工作项，可以向每个工作项负责人发送通知。

JE - Java API 工作区准备

请参阅"工作区准备"章节。

JE - 创建项目插件

JE - 导入示例

信息： 您必须确保您的插件是针对您的 Polarion 版本编译的。此示例包含预编译的 jar 插件。您可以基于此示例开始开发自己的插件之前将其删除。Eclipse 会确保根据您的源代码和 Polarion 版本创建新的 jar 插件。

将工作流项目示例导入工作区的步骤：

1. 选择 File > Import...
2. 在出现的对话框中，选择 General 部分的 Existing Project into Workspace，然后按 Next 按钮。
3. 按 Browse.. 按钮，选择示例目录（通常在 `C:\Polarion\polarion\SDK\examples\` ）。提交。
4. 选择 `com.polarion.example.job` 并按 Finish。

JE - 开发自己的插件提示

首先，您必须创建新的插件项目。填写插件属性并取消选中 Generate activator..。

然后打开 MANIFEST.MF，在 Dependencies 选项卡中将 `com.polarion.alm.tracker`、`com.polarion.platform.jobs` 设置为 Required Plug-in。同样，您应在 Build 选项卡中将 `src/` 文件夹设置为应编译到导出库中的源文件夹。

build.properties 文件内容：

```
### content of 'build.properties' file ###

source.. = src/
output.. = bin/
bin.includes = META-INF/,\
               .
```

JE - 部署到已安装的 Polarion

请参阅"部署到已安装的 Polarion"章节。

JE - 从工作区执行

请参阅"从工作区执行"章节。

JE - 配置

成功将插件部署到 Polarion 后，您可以开始在调度器中使用新任务。要检查部署是否成功，请执行以下步骤：

1. 选择存储库视图。转到管理界面（Administration），选择调度器（Scheduler），您可以在其中添加新任务。
2. 编辑任务的全局配置并为逾期任务示例添加以下行：

任务可以通过属性进行编程，这些属性将由调度器注入到任务实现中。

```
<job name="overdue.job" id="overdue.job" cronExpression="0 0 1 ? * MON-SAT"
    scope="project:playground">
        <query></query>
        <sort>~updated</sort>
        <notificationSender>polarion@example.com</notificationSender>
        <notificationSubjectPrefix>[Polarion]</notificationSubjectPrefix>
        <notificationRecipients>assignee</notificationRecipients>
        <planningConstraint>dueDate</planningConstraint>
        <allowedDelay>0</allowedDelay>
</job>
```

1. 然后您可以切换到项目视图（Projects），选择监控器（Monitor）并监控您的新任务。

6.1.4 通知扩展示例

NEE - 简介

在此示例中，我们将展示如何创建自定义通知目标（Notification Target）。

可以扩展的内容： - 自定义目标 - 在 Administration -> Notifications -> Targets 中扩展配置

示例中将展示的内容： 1. 如何创建自定义目标 - 我们将实现 custom-field-targets 目标，确保通知将发送给在特定自定义字段中找到 ID 的用户。

NEE - Java API 工作区准备

请参阅"工作区准备"章节。

NEE - 创建项目插件

您可以导入已实现的示例，或阅读扩展 Polarion 通知系统所需的步骤。

NEE - 导入示例

信息： 您必须确保您的插件是针对您的 Polarion 版本编译的。此示例包含预编译的 jar 插件。您可以基于此示例开始开发自己的插件之前将其删除。Eclipse 会确保根据您的源代码和 Polarion 版本创建新的 jar 插件。

将工作流项目示例导入工作区的步骤：

1. 选择 File > Import...
2. 在出现的对话框中，选择 General 部分的 Existing Project into Workspace，然后按 Next 按钮。
3. 按 Browse.. 按钮，选择示例目录（通常在 `C:\Polarion\polarion\SDK\examples\` ）。提交。
4. 选择 `com.polarion.example.notifications` 并按 Finish。

NEE - 以自己的方式扩展 Polarion 通知系统

1. 创建新的插件项目。填写插件属性并取消选中 Generate activator..。
2. 在 src 文件夹中创建 META-INF 目录和其中的 hivemodule.xml 文件。
3. 在 hivemodule.xml 中可以设置一个贡献点（Contribution Point）：
    - `com.polarion.psvn.core.notifications.targets` 用于注册新目标
4. 有关语法和更多详细信息，请参阅示例中包含的 hivemodule.xml 文件。
5. 打开 MANIFEST.MF，在 Dependencies 选项卡中将 `com.polarion.alm.tracker`、`com.polarion.platform.persistence`、`com.polarion.psvn.launcher` 设置为 Required Plug-in。同样，您应在 Build 选项卡中将 `src/` 文件夹设置为应编译到导出库中的源文件夹。

build.properties 文件内容：

```
### content of 'build.properties' file ###

source.. = src/
output.. = bin/
bin.includes = META-INF/,\
                .
```

有关如何手动为电子邮件通知设置目标，请参阅文档化示例代码。

NEE - 部署到已安装的 Polarion

请参阅"部署到已安装的 Polarion"章节。

NEE - 从工作区执行

请参阅"从工作区执行"章节。

NEE - 配置

成功将插件部署到 Polarion 后，您可以修改通知配置以开始使用新事件和目标：

1. 选择存储库或项目视图。转到管理界面（Administration），在通知（Notifications）部分选择目标（Targets）。
2. 选择所需的事件。
3. 在目标（Target）下拉菜单中，选择您的自定义通知目标（对于 SDK 为 custom-field-targets）。假设此类字段已定义（请参阅 `com.polarion.example.notifications.targets.CustomFieldTargets` 的文档化示例代码），则在出现的可选字段中输入附加信息。

当您将 custom-field-targets 用于 workitem-commented 事件时，当下新评论被添加到特定工作项时，下拉菜单右侧可选字段中输入的用户 ID 对应的用户将收到通知。**注意：** ID 必须用逗号（,）分隔。

6.1.5 表单扩展示例

FEE - 简介

在此示例中，我们将展示如何创建自定义表单扩展。

可以扩展的内容： - 表单布局 - 我们将演示如何在表单布局中添加您自己的自定义表单扩展

示例中将展示的内容： 1. 如何创建自定义扩展 - 我们将实现一个扩展，显示同一项目中具有相同严重级别的工作项数量。

FEE - Java API 工作区准备

请参阅"工作区准备"章节。

FEE - 创建项目插件

最好的方式是导入提供的示例，其中包含自定义插件创建所需的所有依赖项。

FEE - 导入示例

信息： 您必须确保您的插件是针对您的 Polarion 版本编译的。此示例包含预编译的 jar 插件。您可以基于此示例开始开发自己的插件之前将其删除。Eclipse 会确保根据您的源代码和 Polarion 版本创建新的 jar 插件。

将此示例导入工作区的步骤：

1. 选择 File > Import...
2. 在出现的对话框中，选择 General 部分的 Existing Project into Workspace，然后按 Next 按钮。
3. 按 Browse.. 按钮，选择示例目录（通常在 `C:\Polarion\polarion\SDK\examples\` ）。提交。
4. 选择 `com.polarion.example.formextension` 并按 Finish。

FEE - 部署到已安装的 Polarion

请参阅"部署到已安装的 Polarion"章节。

FEE - 从工作区执行

请参阅"从工作区执行"章节。

FEE - 配置

成功将插件部署到 Polarion 后，您可以修改表单布局配置以开始使用新表单扩展：

1. 选择存储库或项目视图。转到管理界面（Administration），在 Work Items 部分选择表单配置（Form Configuration），在表单布局（Form Layout）部分选择您要显示表单扩展的布局并点击编辑。或者您可以创建一个新的布局。在那里您可以添加以下代码：

```
<extension id="example" label="Example Extension"/>
```

扩展 id 是您在扩展实现中提供的 id：

```
protected void configure() {
        Contributions<FormExtensionContribution> contributions =
new Contributions<FormExtensionContribution>(binder(),
FormExtensionContribution.class);
        contributions.addBinding().toInstance(
new FormExtensionContribution(FormExtensionExample.class, "example"));
    }
```

6.1.6 枚举工厂示例

EFE - 简介

在此示例中，我们将展示如何创建自定义枚举工厂（Enumeration Factory）。

示例中将展示的内容： 1. 如何创建自定义枚举工厂 - 我们将为时间点（Time Point）枚举实现工厂。

EFE - Java API 工作区准备

请参阅"工作区准备"章节。

EFE - 创建项目插件

最好的方式是导入提供的示例，其中包含自定义插件创建所需的所有依赖项。

EFE - 导入示例

信息： 您必须确保您的插件是针对您的 Polarion 版本编译的。此示例包含预编译的 jar 插件。您可以基于此示例开始开发自己的插件之前将其删除。Eclipse 会确保根据您的源代码和 Polarion 版本创建新的 jar 插件。

将此示例导入工作区的步骤：

1. 选择 File > Import...
2. 在出现的对话框中，选择 General 部分的 Existing Project into Workspace，然后按 Next 按钮。
3. 按 Browse.. 按钮，选择示例目录（通常在 `C:\Polarion\polarion\SDK\examples\` ）。提交。
4. 选择 `com.polarion.example.enumerationfactory` 并按 Finish。

EFE - 部署到已安装的 Polarion

请参阅"部署到已安装的 Polarion"章节。

EFE - 从工作区执行

请参阅"从工作区执行"章节。

EFE - 配置

成功将插件部署到 Polarion 后，您可以修改自定义字段配置以开始使用新枚举工厂：

1. 选择存储库或项目视图。转到管理界面（Administration），在 Work Items 部分选择自定义字段（Custom Fields），然后转到特定工作项类型的编辑器。在编辑器中选择类型 "Enum:" 和新选项 "My Time Points"。

6.1.7 管理扩展示例

AEE - 简介

在此示例中，我们将展示如何创建自定义管理页面。

示例中将展示的内容： 1. 如何创建自定义管理页面 2. 如何将自定义管理页面注册到 Polarion

AEE - Java API 工作区准备

请参阅"工作区准备"章节。

AEE - 创建项目插件

最好的方式是导入提供的示例，其中包含自定义插件创建所需的所有依赖项。

AEE - 导入示例

信息： 您必须确保您的插件是针对您的 Polarion 版本编译的。此示例包含预编译的 jar 插件。您可以基于此示例开始开发自己的插件之前将其删除。Eclipse 会确保根据您的源代码和 Polarion 版本创建新的 jar 插件。

将此示例导入工作区的步骤：

1. 选择 File > Import...
2. 在出现的对话框中，选择 General 部分的 Existing Project into Workspace，然后按 Next 按钮。
3. 按 Browse.. 按钮，选择示例目录（通常在 `C:\Polarion\polarion\SDK\examples\` ）。提交。
4. 选择 `com.polarion.example.administration` 并按 Finish。

AEE - 部署到已安装的 Polarion

请参阅"部署到已安装的 Polarion"章节。

AEE - 从工作区执行

请参阅"从工作区执行"章节。

AEE - 配置

自定义管理页面通过 hivemodule.xml 注册到 ID 为 `com.polarion.xray.webui.administrationPageExtenders` 的贡献点。

扩展器属性：id（必需）、name、iconUrl、pageUrl、parentNodeId、parentNodeName、parentNodeIconUrl、projectScope（布尔值）、projectGroupScope（布尔值）、repositoryScope（布尔值）

```
<?xml version="1.0" encoding="UTF-8"?>
<module id="com.polarion.example.administration" version="1.0.0">
 <contribution configuration-
id="com.polarion.xray.webui.administrationPageExtenders">
  <extender id="administrationExample"
  parentNodeId="administrationExampleFolder"
  name="Page" parentNodeName="Example"
  parentNodeIconUrl="/polarion/icons/default/topicIcons/Tools_157-wrench.png"
  iconUrl="/polarion/icons/default/topicIconsSmall/Tools_158-wrench-2.png"
  pageUrl="/polarion/administrationExample/administration.jsp?scope=$scope$"
  projectScope="true" projectGroupScope="true" repositoryScope="true"/>
 </contribution>
</module>
```

6.1.8 自定义导出器示例

CEE - 简介

此导出器允许您测试将工作项导出到 JSON 文件。

CEE - 开发环境

请参阅"环境要求"章节。

CEE - 工作区准备

如果您还没有为 Polarion 插件开发准备的工作区，请在继续本章之前参阅"工作区准备"章节。

您必须首先从 `SDK\example` 目录导入 `com.polarion.example.exporter` 项目。（默认路径为 `Polarion\polarion\SDK\examples\com.polarion.example.exporter`。）

导入 `com.polarion.example.exporter` 项目：

1. 启动 Eclipse，然后选择 File > Import...
2. 在出现的对话框中选择 General 部分的 Existing Projects into Workspace。
3. 点击 Next。
4. 点击 Browse 并选择 `com.polarion.example.exporter` 目录。
5. 点击 OK。
6. 从 Projects 子窗口中选择 `com.polarion.example.exporter` 项目，然后点击 Finish。

CEE - 执行

要了解如何运行/调试带有示例插件的 Polarion，请参阅"部署到已安装的 Polarion"和"从工作区执行"章节。

Polarion 运行后，执行以下步骤查看自定义导出器示例的工作方式：

1. 在 Polarion 中从树状视图（Tree）或表格视图（Table）中选择一个工作项。
2. 点击工具栏中的导出图标。
3. 点击 Export。
4. 打开格式（Format）下拉列表。
5. 如果插件导入成功，您将在列表项中看到 `json: Example Exporter` 选项。
6. 如果该选项未出现在下拉列表中，请检查 `com.polarion.example.exporter` 是否在您 IDE 的运行/调试选项中被选中。
7. 选择 `json: Example Exporter`。
8. （可选）点击 Show Advanced options 以查看导出器的自定义参数。
9. 点击 OK。

CEE - Hivemind 参数

此示例代码展示了如何在代码中定义和使用 Hivemind 参数。在此示例中，我创建了 4 个自定义参数：`max_workitems_for_export`、`write_result_in_german`、`english_result_message` 和 `german_result_message`，它们在插件的 hivemodule.xml 中定义，类型分别为 Integer、Boolean、String 和 String。所有这些类型都是受支持的，参数在代码中的使用可以在 `ExampleExporterCommand.java` 类的不同部分看到。`max_workitems_for_export` 参数限制导出的工作项数量。`write_result_in_german` 参数的效果可以在导出成功完成后在导出对话框中显示的文本中看到。如果参数设置为 true，最终消息将以德语显示（如 `german_result_message` 参数中所定义），否则将以英语显示（如 `english_result_message` 参数中所定义）。

信息： hivemodule 的 exporterDescriptor 部分中的 contentType 是导出内容的 MIME 类型。

CEE - 自定义导出字段

如果您对 Polarion 导出器中默认导出字段的方法和功能不满意，可以创建自己的自定义导出字段。在示例的 `CustomExporterField.java` 类中可以看到一个示例。它实现了 IExportField API 接口，并提供了额外的方法来设置 `readOnly` 和 `columnWidth` 字段。前者在 `ExampleExporterCommand.java` 中用于根据自己的偏好和需求将只读状态应用于字段。

6.2 Web Services 示例

6.2.1 工作项导入示例

IWSE - 简介

工作项导入是一个独立应用程序，展示了一种通过 Web Services 将数据导入到 Polarion 服务器的方法，数据以逗号分隔值（CSV）文件存储，格式如下：

CSV 文件的每行必须有 4 列，由分号或其他在 settings.properties 中定义的项目分隔符分隔。

有序列的含义是：

1. 标题（title）
2. 一行描述（description）
3. 严重级别（severity）（不区分大小写，如果在 Polarion 中找不到，将仅为该工作项定义新的严重级别）
4. 类别（categories）（可以有多个类别，用逗号分隔。类别可以写为类别 ID 或在 Polarion 中定义的类别名称，否则将仅为该工作项定义类别）。

执行结果是关于导入操作成功与否的消息。

IWSE - 使用示例

我们有一些想要导入 Polarion 的数据（在 CSV 文件中）：

```
Filter also administration topics by Hats;description1;Major;Administration
Watch the available disk space;description2;Major;Notifications,defect,Test Suite
```

现在运行以下 shell 脚本命令：

```
import.bat <CSV_File_Path>
```

命令窗口应显示确认工作项已成功导入的消息。

导入的工作项应如下所示。

IWSE - 环境要求

IWSE - 开发环境

- Eclipse IDE
- Eclipse Temurin™ 17 (LTS) by Adoptium

IWSE - Web Services 客户端

您必须使用存储在 Polarion SDK `lib` 目录中的 Polarion Web Services 客户端库（通常在 MS Windows 系统中为 `C:\Polarion\polarion\SDK\lib\com.polarion.alm.ws.client\` ）。

IWSE - 工作区准备

首先，您必须从 `SDK\lib\` 目录导入 Web Services 客户端项目 `com.polarion.alm.ws.client`。执行以下步骤：

1. 启动 Eclipse，然后选择 File > Import...。
2. 在出现的对话框中，选择 General 部分的 Existing Projects into Workspace。按 Next 按钮提交。
3. 按 Browse.. 按钮，选择 `com.polarion.alm.ws.client` 目录。按 OK。
4. 在 Projects 子窗口中选择 `com.polarion.alm.ws.client` 项目并按 Finish。

要开始使用 Web Services，您可以导入示例项目或创建自己的项目。

要导入示例项目，请执行与上述导入 Web Services 客户端项目相同的步骤，但项目名称不同——示例项目位于 `SDK\examples\com.polarion.example.importer`。

要创建您自己的新 Polarion Web Services 项目，请执行以下步骤：

1. 选择 File > New > Java Project。
2. 输入项目名称并点击 Next 按钮。
3. 选择 Projects 选项卡并点击 Add... 按钮。
4. 选择 `com.polarion.alm.ws.client` 项目并提交。
5. 选择 Finish。

IWSE - 执行

您可以在运行（Run）或调试（Debug）模式下执行示例，也可以将其导出为 jar 文件并创建适当的 shell 脚本。

在调试模式下执行：

1. 选择 Run > Open Debug Dialog...
2. 创建新的 Java 应用程序，设置名称，在 Main class 部分点击 Search..。
3. 在新的弹出窗口中选择 `com.polarion.example.importer.Importer` 类。
4. 在第二个选项卡 - Arguments 中，在 Program arguments 部分设置 CSV 文件的名称（或路径）。
5. 点击 Debug 按钮。

导出为 jar 归档：

1. 选择 File > Export...
2. 在 Java 部分选择 JAR file，点击 Next...

3. 勾选我们的项目（例如此示例）并设置新 JAR 归档的位置。
4. 点击 Finish 按钮。
5. 要运行 jar，您可以使用示例文件夹中存储的 MS Windows 脚本 `import.bat`。

IWSE - 配置和使用

要开始使用示例，您必须在 settings.properties 文件中设置有关 Polarion 服务器的信息。

IWSE - settings.properties

此文件必须存储在与编译项目相同的目录级别。格式为：`key=value`。必须设置以下键：

- `polarion_server_address`，例如 `'http://localhost'`
- `polarion_server_port`，例如 `'81'`
- `user`，例如 `'admin'`
- `passwd`，例如 `'admin'`
- `project_id`，例如 `'requ'`（需求项目）
- `module`，例如 `'Playground Module'` - 默认是将工作项导入到任何模块之外
- `item_delimiter`，例如 `';'`
- `wi_type`，例如 `'requirement'` - 默认值设置为 "requirement"，所有导入的工作项将具有 'wi_type' 类型

IWSE - CSV 数据示例：

```
Filter also administration topics by Hats;description1;Major;User Management,
Administration
Watch the available disk space;description2;Major;Backend, User_man
```

在 Polarion 中定义的术语： - 严重级别 Major（id=major, name=Major） - 类别 User Management（id=User_man, name=User Management）、Backend（id=Backend, name=Backend）

6.2.2 提交前钩子示例

PHE - 简介

PHE - 关于提交前钩子（Pre-commit hook）

提交前钩子在 Subversion（SVN）事务提交之前调用。Subversion 通过调用名为 'pre-commit' 的程序（脚本、可执行文件、二进制文件等）来运行此钩子，参数按以下顺序传递：

1. REPOS-PATH（此存储库的路径）
2. TXN-NAME（要提交的事务名称）

如果钩子程序成功退出，则事务被提交；但如果以失败（非零）退出，则事务被中止，不会进行提交，且 STDERR 返回给客户端。

更多信息请参阅：钩子脚本（hook scripts）。

PHE - 关于此示例应用程序

此钩子运行 Java 应用程序，该程序将检查提交消息的注释中是否包含可解析且未解决的工作项链接。目的是确保每个修订都将链接到未解决的工作项。

如果提交消息中的工作项链接有效，则应用程序的返回代码等于 0。您应该设置错误日志文件的路径。请参阅"配置和使用"章节了解配置文件。

PHE - 使用示例

展示了以下几种情况： - 未设置任何消息的情况 - 设置了正确消息且工作项存在但未解决的情况 - 设置了正确消息但工作项已解决（状态为 'Closed'）的情况

PHE - 环境要求

PHE - 开发环境

- Eclipse IDE
- Eclipse Temurin™ 17 (LTS) by Adoptium

PHE - Web Services 客户端

您必须使用存储在 Polarion SDK `lib` 目录中的 Polarion Web Services 客户端库（通常在 MS Windows 系统中为 `C:\Polarion\polarion\SDK\lib\com.polarion.alm.ws.client\` ）。

PHE - 工作区准备

首先，您必须从 `SDK\lib\` 目录导入 Web Services 客户端项目 `com.polarion.alm.ws.client`。执行以下步骤：

1. 启动 Eclipse，然后选择 File > Import...。
2. 在出现的对话框中，选择 General 部分的 Existing Projects into Workspace。按 Next 按钮提交。
3. 按 Browse.. 按钮，选择 `com.polarion.alm.ws.client` 目录。按 OK。
4. 在 Projects 子窗口中选择 `com.polarion.alm.ws.client` 项目并按 Finish。

要开始使用 Web Services，您可以导入示例项目或创建自己的项目。

要导入示例项目，请执行与上述导入 Web Services 客户端项目相同的步骤，但项目名称不同——示例项目位于 `SDK\examples\com.polarion.example.commithook`。

要创建您自己的新 Polarion Web Services 项目，请执行以下步骤：

1. 选择 File > New > Java Project。

2. 输入项目名称并点击 Next 按钮。
3. 选择 Projects 选项卡并点击 Add... 按钮。
4. 选择 `com.polarion.alm.ws.client` 项目并提交。
5. 选择 Finish。

PHE - 执行

要执行示例，您必须将其导出为 jar 文件并创建适当的 shell 脚本。执行以下步骤：

1. 选择 File > Export...
2. 在 Java 部分选择 JAR file，点击 Next…
3. 勾选我们的项目（此示例 = `com.polarion.example.commithook`）并设置新 JAR 归档的位置（包括新 jar 归档的名称，例如 `pre-commit.jar`）。
4. 点击 Finish 按钮。

要运行 jar，您可以使用 Windows 脚本：（参见提交钩子示例文件夹中的 pre-commit.bat 文件）

您需要做的是：

- 将此脚本保存到您的 SVN 钩子目录（例如 Polarion SVN 钩子目录 = `C:\Polarion\data\svn\repo\hooks\`）。
- 设置 POL_PROP 变量指向配置文件，其中设置了 Polarion 服务器地址、登录名、密码等。详细信息请参阅"配置和使用"章节。
- 将 PRE_COMMIT_JAR 变量设置为您在上一步中导出项目的相同位置。
- 将 JAVA_HOME 变量设置为您用于编译此项目的相同 Java 环境。
- 根据您的系统和 Polarion 安装设置 POLARION_SDK_DIR 变量。

PHE - 配置

要开始使用示例，您必须在 settings.properties 文件中设置有关 Polarion 服务器的信息。

PHE - settings.properties

格式为：`key=value`。必须设置以下键：

- `polarion_server_adr`，例如 `'http://localhost'` - Polarion 服务器地址
- `polarion_server_port`，例如 `'81'` - Polarion 服务器端口
- `user`，例如 `'admin'` - 登录 Polarion 的用户名
- `passwd`，例如 `'admin'` - 'user' 的密码
- `project_id`，例如 `'requ'`（需求项目）- 提交消息中键入的 WorkItem 所属项目的 ID
- `svnlook_dir`，例如 `'C:\\Polarion\\bundled\\svn\\bin\\'` - svnlook 所在目录的路径
- `svnlook_cmd`，例如 `'svnlook.exe'` - svnlook 程序文件名

- `apache_log_folder`，例如 `'C:\\Polarion\\data\\logs\\apache\\'` - 错误日志应放置的目录路径
- `apache_log_file_name`，例如 `'commit_audit.log'` - 错误日志文件名称（验证提交消息期间产生的错误）

信息： 此示例是为 MS Windows 系统开发的，对于不同的操作系统，请在源代码中更改路径和路径分隔符。

---

7 Polarion Java API 文档（用于 Polarion 扩展）

请参阅 JavaDoc 页面（在外部 Web 查看器中打开）。

8 Hivedoc

请参阅 HiveDoc 页面（在外部 Web 查看器中打开）。

9 Web Services 文档

Polarion 服务器可用的 Web Services 列表：

BuilderWebService

提供与构建（Builds）相关的功能（例如项目构建列表）。 - 查阅 BuilderWebService 接口的 Web Services 客户端 Javadoc（在外部 Web 查看器中打开） - 查阅 BuilderWebService.wsdl 的源代码（在外部查看器中打开）

ProjectWebService

提供与项目相关的功能（例如哪些用户参与了某个项目）。 - 查阅 ProjectWebService 接口的 Web Services 客户端 Javadoc（在外部 Web 查看器中打开） - 查阅 ProjectWebService.wsdl 的源代码（在外部查看器中打开）

SecurityWebService

提供安全相关信息，主要关注用户权限。 - 查阅 SecurityWebService 接口的 Web Services 客户端 Javadoc（在外部 Web 查看器中打开） - 查阅 SecurityWebService.wsdl 的源代码（在外部查看器中打开）

SessionWebService

提供与 Web Services 当前会话相关的功能，主要关注会话管理（登录、显式事务等）。 - 查阅 SessionWebService 接口的 Web Services 客户端 Javadoc（在外部 Web 查看器中打开） - 查阅 SessionWebService.wsdl 的源代码（在外部查看器中打开）

TestManagementWebService

提供与测试运行（Test Runs）相关的功能（例如项目测试运行列表）。 - 查阅 TestManagementWebService 接口的 Web Services 客户端 Javadoc（在外部 Web 查看器中打开） - 查阅 TestManagementWebService.wsdl 的源代码（在外部查看器中打开）

TrackerWebService

提供与追踪器相关的功能，如创建工作项（Work Items）、评论（Comments）等，添加和移除工作项属性（修订、负责人、类别等），执行查询或获取可用操作。 - 查阅 TrackerWebService 接口的 Web Services 客户端 Javadoc（在外部 Web 查看器中打开） - 查阅 TrackerWebService.wsdl 的源代码（在外部查看器中打开）

---

10 数据库

Polarion 的架构包含一个数据库，对于某些类型的报告，可以比 Lucene 查询引擎更高效地查询。数据库文件夹的索引页提供对各种资源的访问，包括模式图（Schema Diagrams）、远程连接信息和表参考（Tables Reference）。

11 自定义字段类型的 Java 类

信息： 如何创建以下每个实例的示例可以在它们的 Javadoc 中找到。

人类可读的自定义字段类型名称	内部自定义字段类型名称	Java 类
String（单行纯文本）	string	Java.lang.String
Text（多行纯文本）	text/plain	com.polarion.core.util.types.Text
Rich Text（多行富文本）	text/html	com.polarion.core.util.types.Text

人类可读的自定义字段类型名称	内部自定义字段类型名称	Java 类
Integer（整数）	integer	java.lang.Integer
Boolean（布尔值）	boolean	java.lang.Boolean
Float（浮点数）	float	java.lang.Float
Date time（日期时间）	date-time	java.util.Date
Date（日期）	date	com.polarion.core.util.types.DateOnly
Duration（持续时间）	duration	com.polarion.core.util.types.duration.DurationTime
Time（时间）	time	com.polarion.core.util.types.TimeOnly
Currency（货币）	currency	com.polarion.core.util.types.Currency